KherveCAD: an open parametric modeller that language-model agents can drive

Gwilherm Kerherve[a,*]

[a]*Department of Materials, Imperial College London, Royal School of Mines, South Kensington, London SW7 2BP, United Kingdom*

Abstract

KherveCAD is a desktop modeller written in Python and Qt that keeps every design as a tree of objects, and that tree compiles one-to-one into an OpenSCAD program. Because the tree and the program cannot drift apart, a part drawn with the mouse can be edited as code and a program pasted in comes back as editable objects. The application also exposes its editing operations through the Model Context Protocol (MCP). A language-model assistant connected this way does not stop at writing code: it can look at a render of what it built, measure volumes, test a part for 3D printing and check whether two parts collide, and then correct its own work. We describe the architecture, give three worked examples (a printable hinge, an ultra-high-vacuum chamber layout and a carbon nanotube), and discuss where the tool has been useful in teaching and laboratory design.

Keywords: computer-aided design, OpenSCAD, Model Context Protocol, large language models, parametric modelling, 3D printing

Code metadata

1. Motivation and significance

Scientists often need small custom parts: a sample holder, a flange adapter, a bracket for an optical bench. Commercial CAD packages handle this well but are expensive and slow to learn, so many researchers turn to OpenSCAD [1] instead. OpenSCAD describes a solid as a short program, which

*Corresponding author

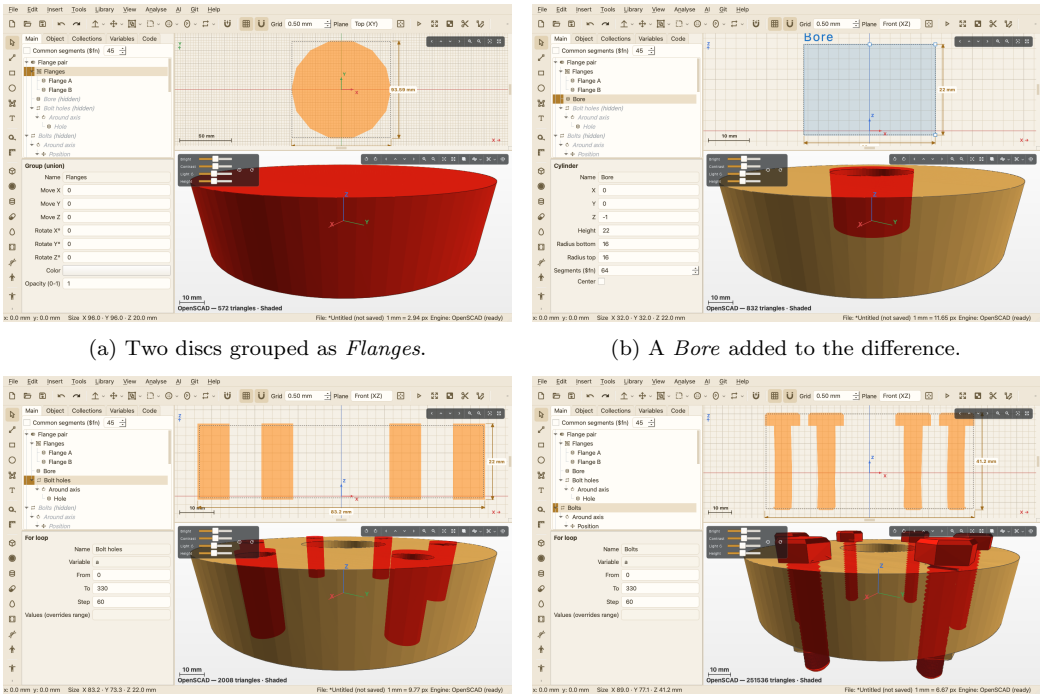
Email address: g.kerherve@ic.ac.uk (Gwilherm Kerherve)

Nr	Code metadata description	Value
C1	Current code version	v0.1.546
C2	Permanent link to code/repository	`https://github.com/gkerherve/KherveCAD`
C3	Permanent link to reproducible capsule	[Zenodo DOI]
C4	Legal code licence	GPL-3.0
C5	Code versioning system used	git
C6	Software code languages, tools and services used	Python 3.11+, PyQt5, OpenSCAD (optional), manifold3d, numpy
C7	Compilation requirements, operating environments and dependencies	Windows 10/11, macOS (Apple silicon and Intel); see `requirements.txt`
C8	Link to developer documentation/-manual	Built-in user guide (F1); `docs/MCP.md`
C9	Support email for questions	g.kerherve@ic.ac.uk

makes a design easy to version, share and tweak by changing a number. The price is that everything must be typed. Nothing can be dragged, and a beginner has to picture the shape in their head before seeing it.

Language models have changed this picture somewhat. Several groups have shown that an LLM can produce CAD from a description, either as sketch-and-extrude sequences [2, 3] or as scripts in CadQuery and similar languages [4, 5]. In practice the code often compiles but the part is wrong — a hole on the wrong face, two pieces that overlap, a wall too thin to print — and the model has no way of knowing. Alrashedy et al. [6] showed that letting a vision-language model inspect renders of its output and revise the code improves results, and Sadik and Bujny [7] argue that geometric metrics are needed to judge such output at all.

KherveCAD was built to close both gaps. On the human side it gives OpenSCAD a graphical editor in which shapes are drawn, moved and combined by hand, while the program stays visible and editable in a code tab. Fig. 1 shows a flange joint built this way, one node at a time, and Fig. 2 the finished part. On the machine side it offers the same editor to an AI assistant through the Model Context Protocol [8], an open standard for connecting assistants to tools. The assistant gets the operations a person would use, plus the means to inspect the result. Our view is that this inspection step, more than code generation itself, is what makes AI-assisted design dependable



(a) Two discs grouped as *Flanges*.

(b) A *Bore* added to the difference.

(c) A `for` loop places six bolt holes.

(d) Bolts and nuts, two more loops.

Figure 1: Building a bolted flange joint in the main window. Each step adds one node to the tree on the left. The selected node is tinted red in the 3D view, so in (b) and (c) the red solids are the material the difference removes.

enough for laboratory work.

2. Software description

2.1. Architecture

The whole program rests on one rule: the object tree is the only copy of the design (Fig. 3). Each node — a cube, an extrusion, a translation, a boolean difference, a loop — corresponds to exactly one OpenSCAD statement. The code shown to the user is regenerated from the tree after every change, and code typed by the user goes through a parser that rebuilds the tree. A round trip from tree to code and back is lossless, and the test suite checks it.

Geometry is shown in two stages. A tessellator written in Python draws an immediate preview, with boolean operations cut by the Manifold library [9] when it is installed. OpenSCAD then renders each part separately

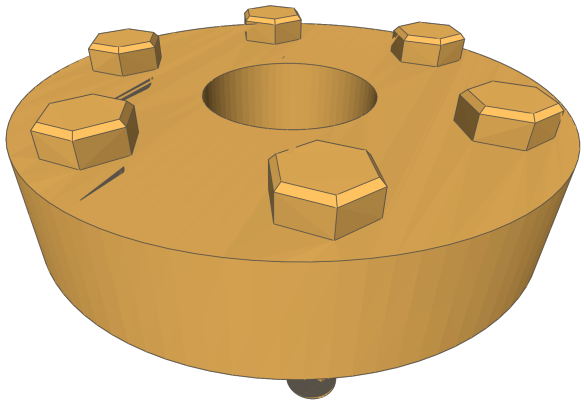


Figure 2: The finished flange joint, rendered exactly by OpenSCAD.

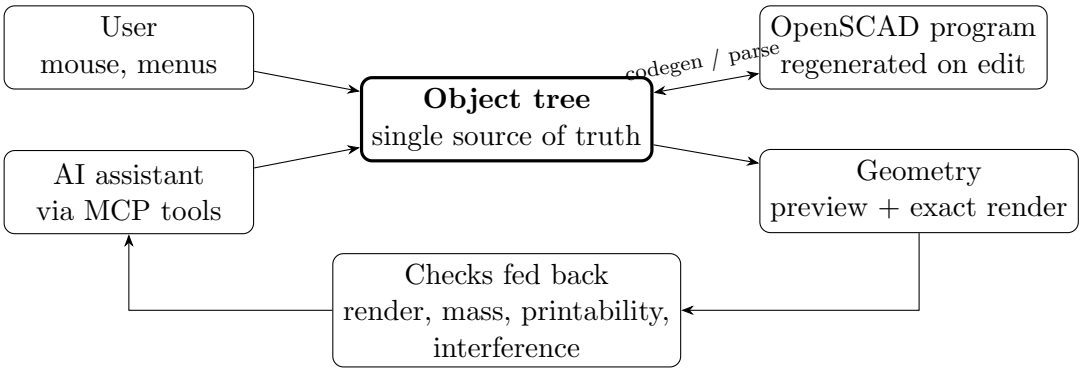


Figure 3: Architecture. The mouse and the assistant edit the same tree; the assistant also receives the results of the checks.

in the background, and the exact mesh replaces the preview as it arrives. Rendering per part rather than per document keeps each part's colour and means that moving one part re-renders nothing.

Parts are organised the way most engineering CAD does it. An *Object* is a part definition that compiles to an OpenSCAD `module`, and the main tab is an assembly of placed instances. Instances can be mated face to face (flush, concentric, hinge, gear ratio), and the program still reads as plain OpenSCAD.

2.2. The assistant interface

The MCP server exposes 88 tools. Rather than describe them all, it helps to say what kind they are. Some edit the tree directly, such as adding a node

or setting a parameter; the most used, `apply_code`, takes an OpenSCAD program and parses it into real nodes, so the user can still drag a corner afterwards. Others only read. `render_view` returns an image of the model from a chosen angle, so the assistant sees what it made. `check_printability` reports open edges, overhangs and thin walls. `mass_properties` and `check_interference` give numbers it can compare against the brief.

The server listens only on the local machine and needs a per-session token. The user picks how much the assistant may do: read only, edit the open document, or also read and write files. Each tool call becomes a single undo step, so the person at the keyboard can reverse anything the assistant did with Ctrl+Z.

2.3. Other functions

Around this core sit tools that grew from our own needs. These include fillets picked on an edge, shelling, patterns, sculpting, sheet metal with flat patterns, 2D engineering drawings and STEP export through Open CASCADE [10]. There is also a part library, which ranges from ISO fasteners and CF vacuum flanges to laboratory glassware, as well as builders for crystal structures, surfaces cut along any (hkl), molecules from SMILES and peptides.

3. Illustrative examples

A butt hinge in three printable parts. The hinge is three Objects: *Leaf A*, *Leaf B* and a removable threaded pin, *Bolt* (Fig. 4). In the tree each one is a single row with its own colour and placement. In the program each one is an OpenSCAD `module` followed by one placed call (Listing 1). Clearances between knuckles and around the pin are named variables, so changing the printer tolerance is a one-line edit. The shipped file is generated by a script, and a test renders it with OpenSCAD to confirm that every piece is watertight and that the leaves swing a full 180° without touching.

An ultra-high-vacuum chamber. The Chamber Designer takes a chamber body and a list of ports, each aimed at a focal point on the axis, and fills them with flanges and instruments from the library (Fig. 5). It reports clashes between ports in 3D before anything is ordered. All port-mounted parts follow one convention — sealing face at $z = 0$, vacuum side down — so any instrument fits any port of the right flange size.

Listing 1: Outline of the 216-line program KherveCAD generates for the hinge. Each Object becomes a module and one placed call. The body of `Bolt` is shown in full, with long point lists cut.

```
module Leaf_A() { ... } // 97 lines
color("#d1ad59") Leaf_A(); // Leaf A
module Leaf_B() { ... } // 74 lines
color("#d1ad59") Leaf_B(); // Leaf B
module Bolt() {
    difference() {
        union() {
            translate([0, 0, 4.6]) rotate([0, 90, 0]) // head
                rotate_extrude($fn=96) polygon(points=[...]);
            translate([-0.5, 0, 4.6]) rotate([0, 90, 0]) // shank
                cylinder(h=59, r1=4.95, r2=4.95, $fn=96);
            translate([58, 0, 4.6]) rotate([0, 90, 0]) // thread
                linear_extrude(height=14.5, twist=-2610, slices=174)
                    polygon(points=[...]);
        }
        translate([-7, -10, -10]) cube([80.5, 20, 10]); // flat to print on
        translate([-6.01, 0, 4.6]) rotate([0, 90, 0]) // hex socket
            cylinder(h=5.01, r1=3.4641, r2=3.4641, $fn=6);
        translate([0, 0, 4.6]) rotate([0, 90, 0]) // chamfers
            rotate_extrude($fn=96) polygon(points=[...]);
    }
}
color("#a8843f") translate([0, 0, 4.4]) Bolt();
```

A carbon nanotube. Carbon nanostructures are built on their lattice rather than from a list of coordinates. An (n, m) nanotube is rolled from a graphene strip along the chiral vector, so the seam closes exactly and the axis stays on z; graphene sheets, nanoribbons and fullerenes come from the same module. Fig. 6 shows an $(8, 8)$ armchair tube 4 nm long in ball-and-stick style. The document switches to nanometres on insertion, and the structure remains an editable group of atoms and bonds. The same structures are available to an assistant through `insert_part`. [Describe here a real assistant session: the request, the tool calls and any correction it made after looking at the render.]

4. Impact

The first use was in our own laboratory, where chambers, sample holders and adapters had been drawn in a mix of commercial CAD and hand sketches. Keeping them as short programs in one repository made them easier to revise when a port moved or a flange size changed. [Add concrete cases.]

The second use is teaching. The 21 tutorials included in the application take a student from a single cube to loops and modules, and because the

code tab follows every click, students see the program grow as they draw. [Add course, number of students, feedback.]

The third, and probably the most interesting for other researchers, is as a test bed for AI-assisted design. Most published text-to-CAD work evaluates code that was generated once and then compiled [3, 5]. KherveCAD makes it possible to study the loop instead: an agent that builds, looks, measures and corrects, in the spirit of [6] but with checks that go beyond the image. The same tools are available to any MCP client, so different models can be compared on identical tasks without writing a new harness for each.

The software has been publicly available since [date]. [Add downloads, stars, external users.]

Limitations. The built-in preview approximates some operations until OpenSCAD has rendered them. Sketches have no geometric constraint solver yet. The assistant's results also depend on the language model behind it; the tools make errors visible but do not prevent them.

5. Conclusions

KherveCAD puts a graphical editor on top of OpenSCAD without hiding the code, and it lets an AI assistant use that editor with the same operations a person has. The design choice that matters most is the single tree behind both. Next steps are a sketch constraint solver and a public benchmark of design tasks for comparing language models inside the tool.

Declaration of competing interest

The author declares no competing financial interests or personal relationships that could have influenced the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude (Anthropic) to help write code and draft text. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Acknowledgements

[To fill in.]

References

[1] M. Kintel, C. Wolf, OpenSCAD: The programmers' solid 3D CAD modeller, `https://openscad.org`, accessed 12 October 2026.

[2] S. Wu, A. Khasahmadi, M. Katz, et al., CAD-LLM: Large language model for CAD generation, in: NeurIPS 2023 Workshop on Machine Learning for Creativity and Design, 2023.

[3] M. S. Khan, S. Sinha, T. U. Sheikh, D. Stricker, S. A. Ali, M. Z. Afzal, Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts, in: Advances in Neural Information Processing Systems, Vol. 37, 2024. `doi:10.52202/079017-0242`.

[4] H. Xie, F. Ju, Text-to-CadQuery: A new paradigm for CAD generation with scalable large model capabilities (2025). `arXiv:2505.06507`.

[5] Y. Guan, X. Wang, X. Xing, J. Zhang, D. Xu, Q. Yu, CAD-Coder: Text-to-CAD generation with chain-of-thought and geometric reward, in: Advances in Neural Information Processing Systems, 2025, arXiv:2505.19713.

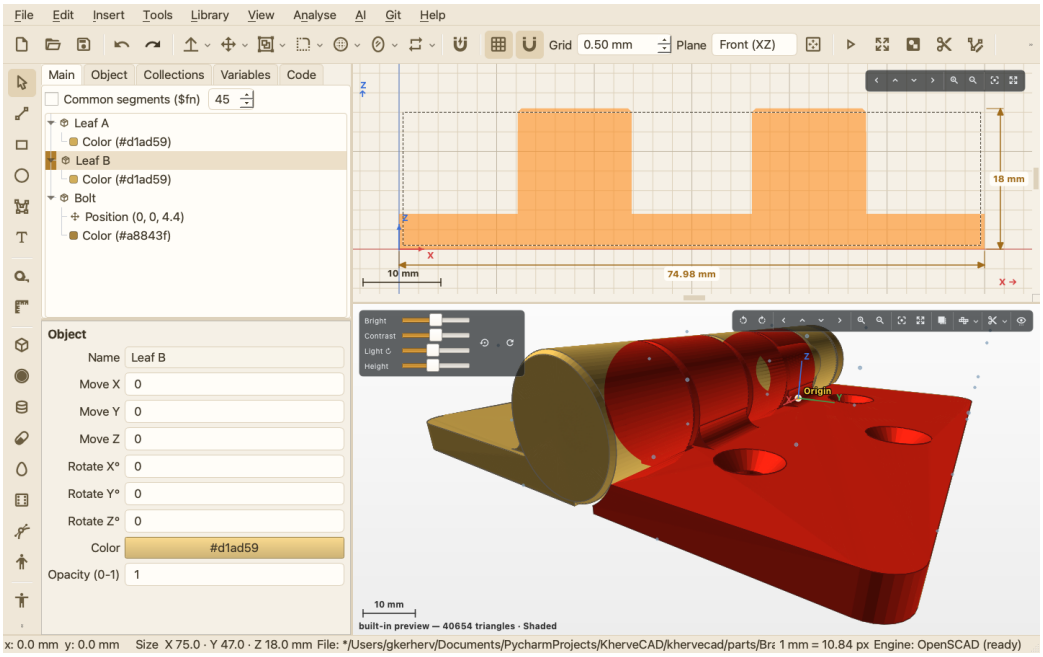
[6] K. Alrashedy, P. Tambwekar, Z. Zaidi, M. Langwasser, W. Xu, M. Gombolay, Generating CAD code with vision-language models for 3D designs, in: International Conference on Learning Representations (ICLR), 2025, arXiv:2410.05340.

[7] A. R. Sadik, M. Bujny, Human-in-the-loop: Quantitative evaluation of 3D models generation by large language models (2025). `arXiv:2509.07010`.

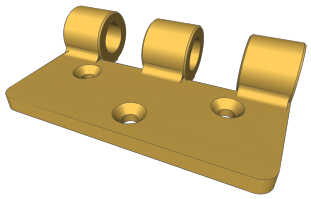
[8] Anthropic, Model context protocol specification, `https://modelcontextprotocol.io`, accessed 12 October 2026 (2024).

[9] E. Lalish, et al., Manifold: a geometry library for topological robustness, `https://github.com/elalish/manifold`, accessed 12 October 2026.

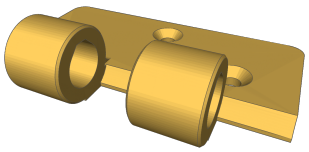
[10] Open Cascade SAS, Open CASCADE technology, `https://dev.opencascade.org`, accessed 12 October 2026.



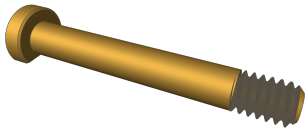
(a) The tree holds three Objects; Leaf B is selected.



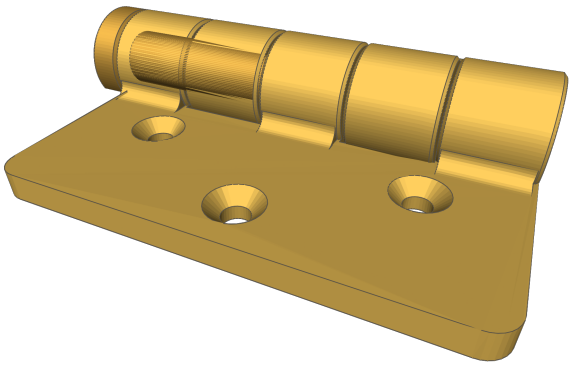
(b) Leaf A



(c) Leaf B



(d) Bolt (pin)



(e) Assembled

Figure 4: Butt hinge: the object tree, each printable part on its own, and the assembly.

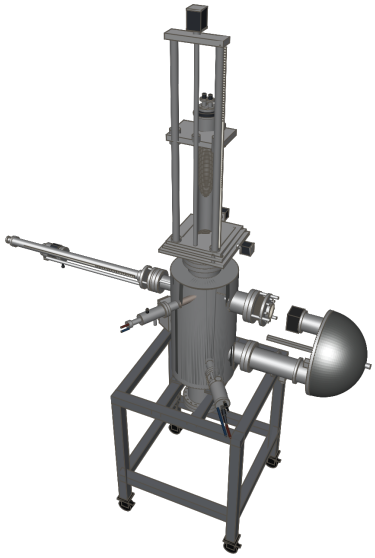


Figure 5: Two-level chamber from the Chamber Designer: manipulator on top, transfer arm, hemispherical analyser and source ports, on a frame with castors.

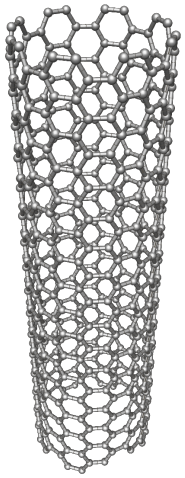


Figure 6: $(8, 8)$ armchair carbon nanotube, 4 nm long, ball-and-stick.